ML for Mech Eng (CoEP)

Yogesh Haribhau Kulkarni

YHK

Outline

YHK

About Me

YHK

Yogesh Haribhau Kulkarni

Bio:

- 20+ years in CAD/Engineering software development
- Got Bachelors, Masters and Doctoral degrees in Mechanical Engineering (specialization: Geometric Modeling Algorithms).
- Currently doing Coaching in fields such as Data Science, Artificial Intelligence Machine-Deep Learning (ML/DL) and Natural Language Processing (NLP).
- Feel free to follow me at:
  - Github (github.com/yogeshhk)
  - LinkedIn (www.linkedin.com/in/yogeshkulkarni/)
  - Medium (yogeshharibhaukulkarni.medium.com)
  - Send email to yogeshkulkarni at yahoo dot com

Office Hours: Saturdays, 2 to 5pm (IST); Free-Open to all; email for appointment.

YHK

SVM

Support Vector Machines (SVM)

- Support vector machines are supervised machine learning algorithms that construct separating planes for classification.
- They are conceptually fairly simple, but the underlying mathematics for learning them from data can be tricky.

YHK

Support Vector Machines (SVM) Inventors

- See video "Vladimir Vapnik - 2012 Laureate of the Franklin Institute in Computer and Cognitive Science" at Youtube.
- Vladimir Vapnik discusses how his early years in Moscow helped shaped the origins of statistical learning theories, including the ideas behind the SVM.
- His collaborator, Isabelle Guyon, also discusses the Kernel Trick, which generalized the SVMs for non-linear decision boundaries (hence making them more popular).

YHK

Support Vector Machines (SVM) Inventors

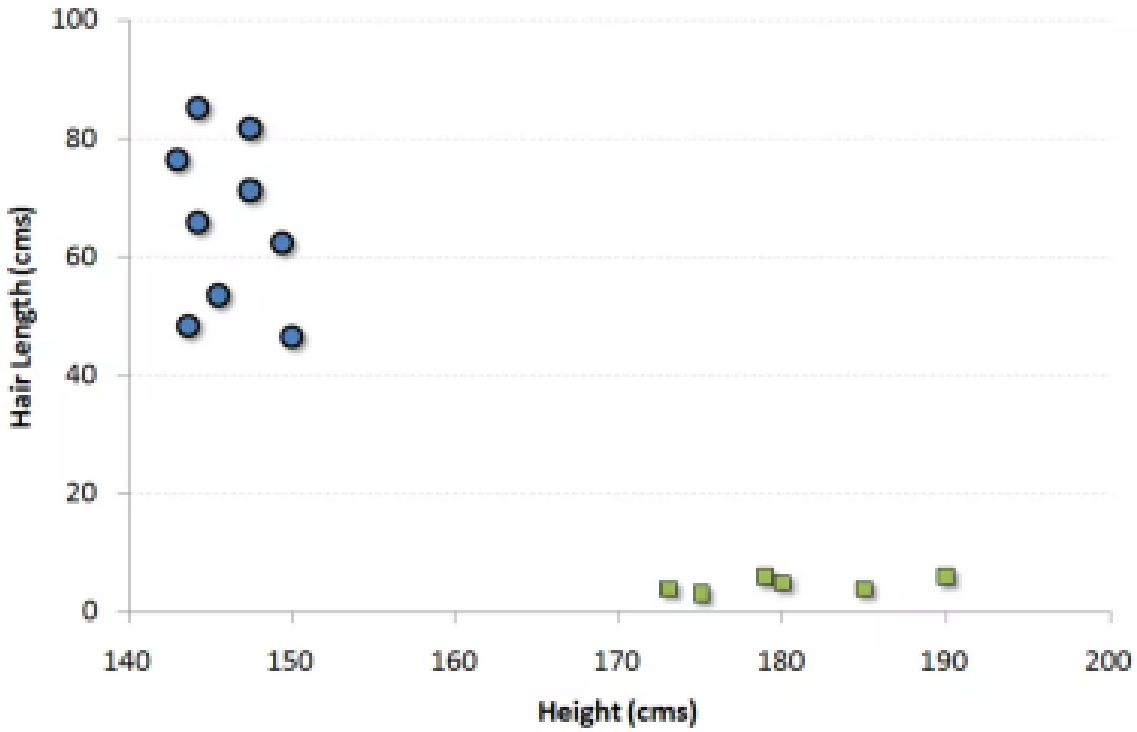
"The invention of SVMs happened when Bernhard decided to implement Vladimir's algorithm in the three months we had left before we moved to Berkeley. After some initial success of the linear algorithm, Vladimir suggested introducing products of features. I proposed to rather use the kernel trick of the 'potential function' algorithm. Vladimir initially resisted the idea because the inventors of the 'potential functions' algorithm (Aizerman, Braverman, and Rozonoer) were from a competing team of his institute back in the 1960's in Russia!" - Isabelle Guyon

YHK

Support vector machines Example

- In SVM, we plot each data item as a point in n dimensional space (where n is number of features you have) with the value of each feature being the value of a particular coordinate.
- For example, if we only had two features like Height and Hair length of an individual.
- We'd first plot these two variables in two dimensional space where each point has two-coordinates.

YHK



Now, we will find some line that splits the data between the two differently classified groups of data.

YHK

Support Vector Machines (SVM)

SVMs are the combination of two ideas

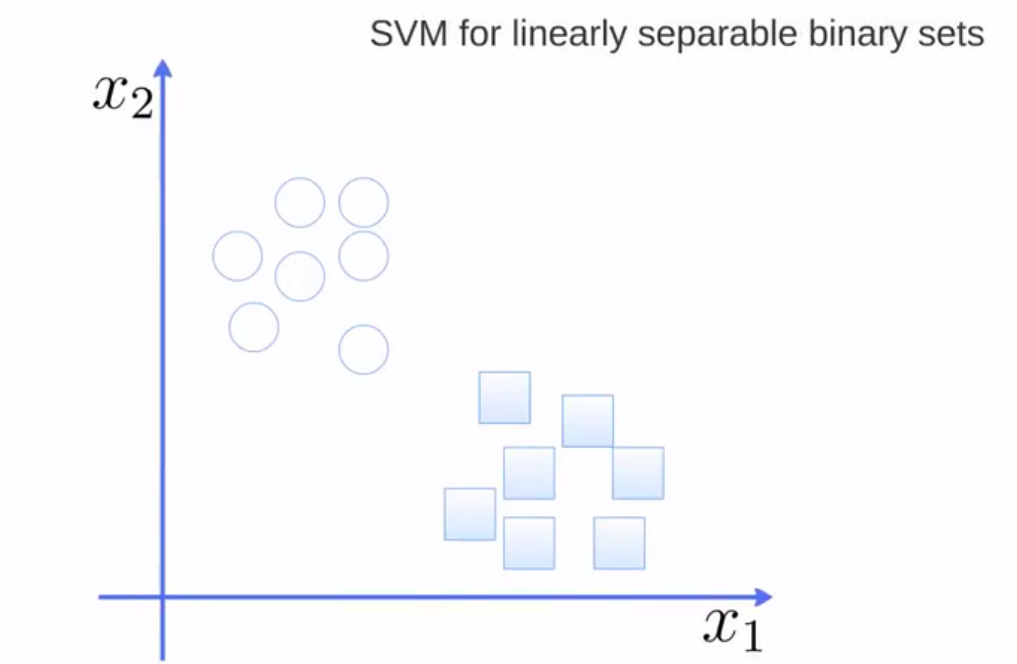
- Maximum margin classification
- The “kernel trick”

YHK

SVM Intuition

YHK

SVM Intuition

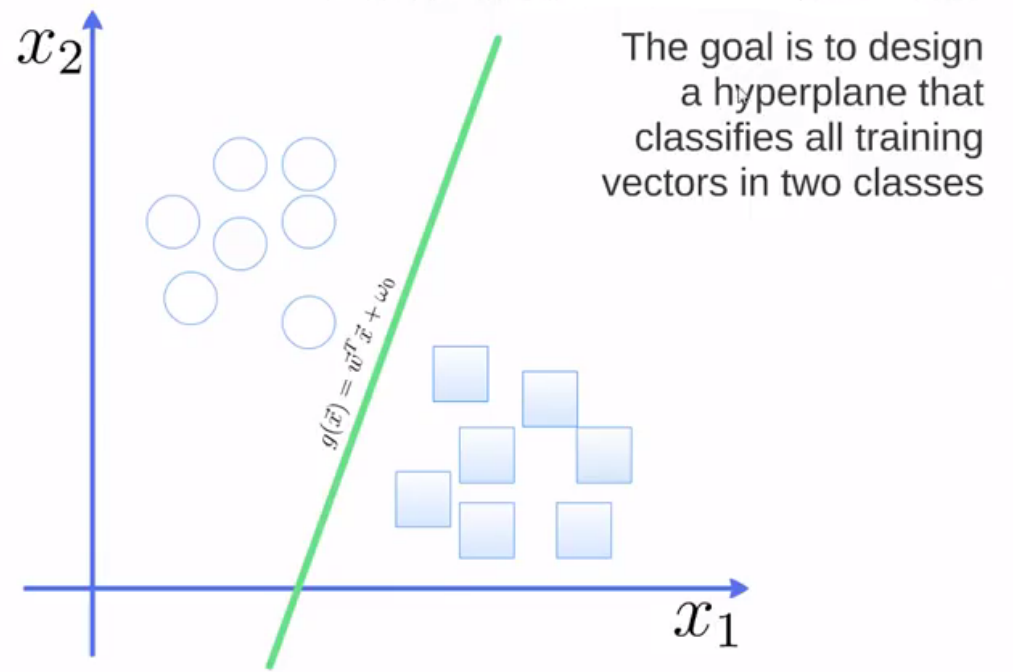


(Ref: How Classification Algorithms Work? - Thales Sehn Korting)

- Suppose there are points characterized by x_1, x_2.
- So these are features
- Points are labeled with either of two classes: circles and squares.

YHK

SVM Intuition

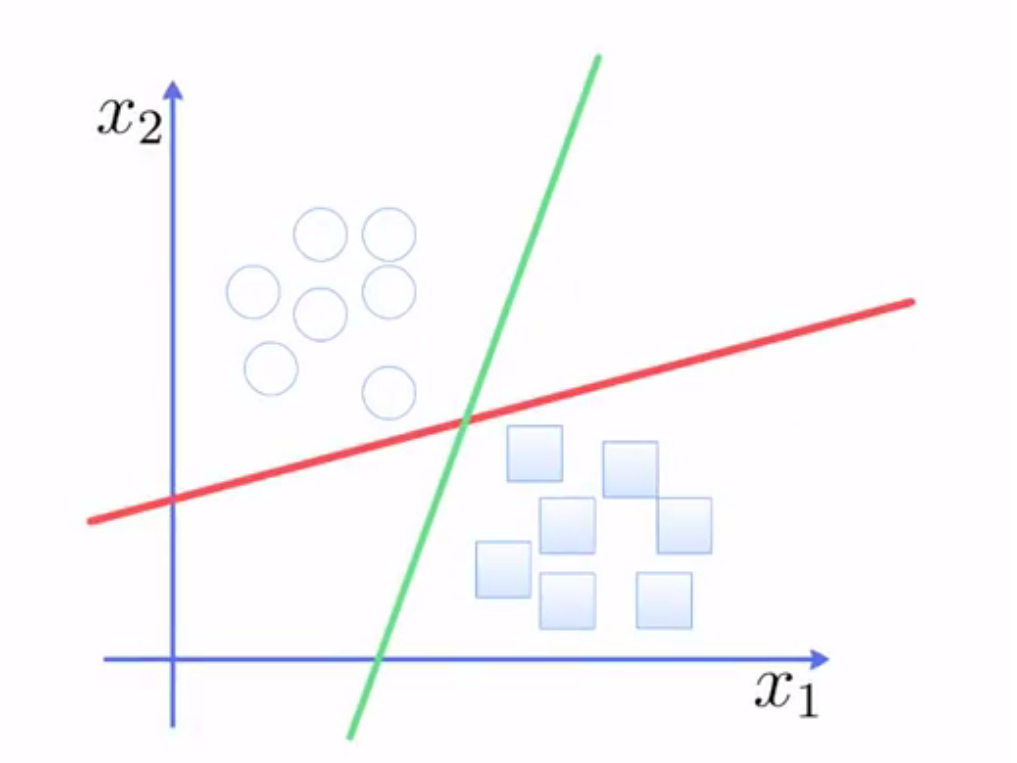


(Ref: How Classification Algorithms Work? - Thales Sehn Korting)

- Goal is to find a separating plane that separates the classes as per the training data
- Generic Hyper-plane's equation is taken as $g(\overrightarrow{x}) = \overrightarrow{w}^T\overrightarrow{x} + w_0$

YHK

SVM Intuition

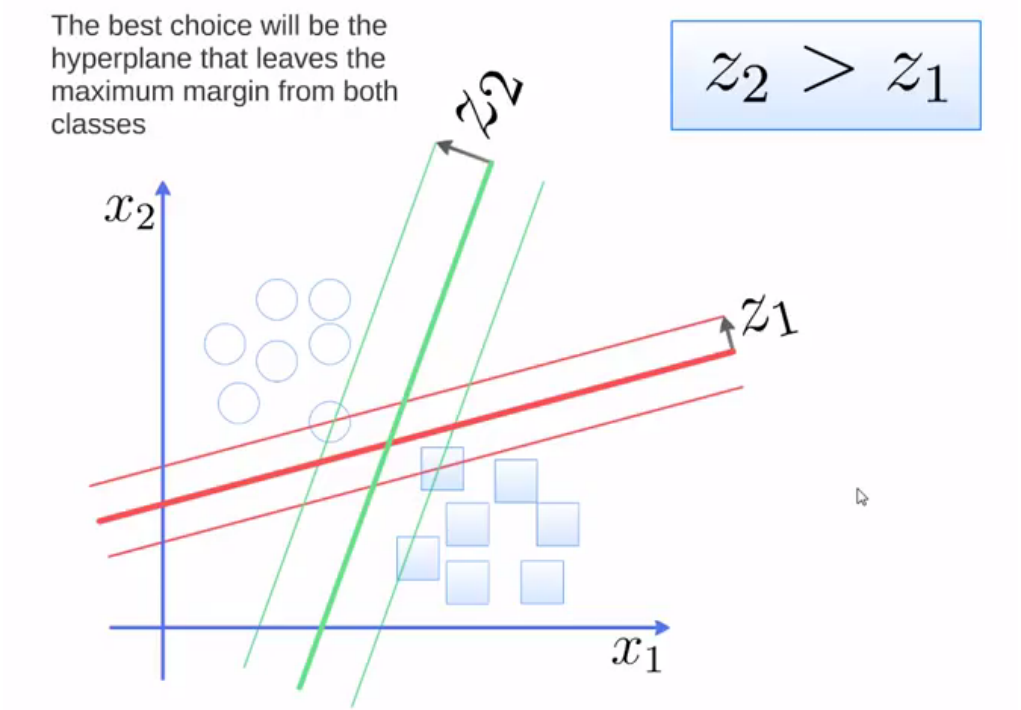


(Ref: How Classification Algorithms Work? - Thales Sehn Korting)

- Many Hyper-planes can classify the data correctly.
- But which one is the best?

YHK

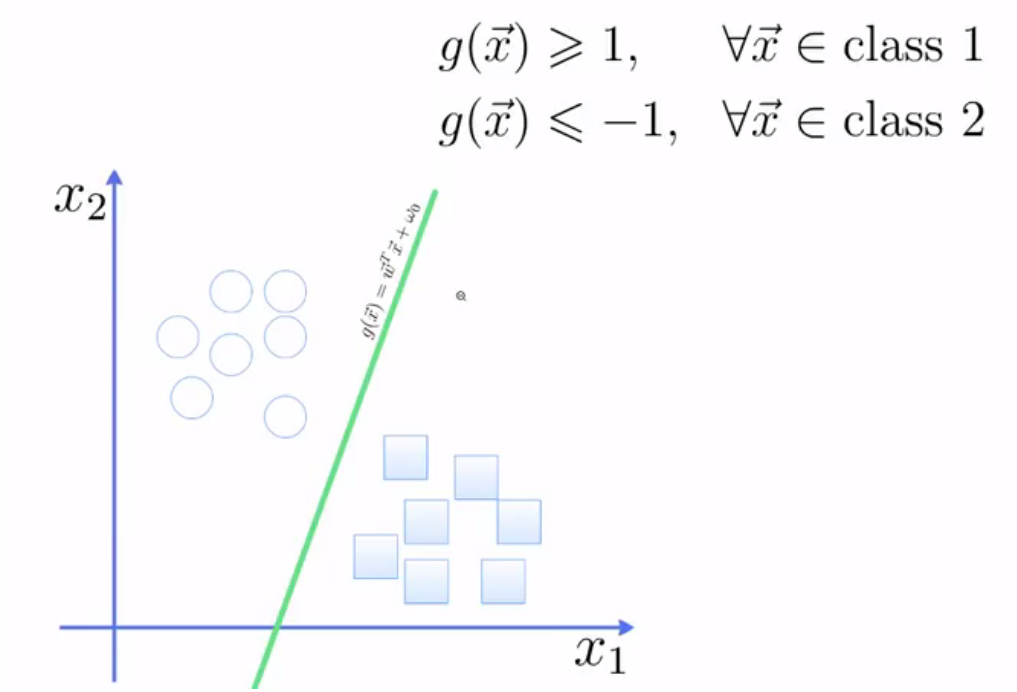
SVM Intuition



- The one with Maximum margin is the best.
- Z_2 is better than Z_1, so nice and distinct separation.

YHK

SVM Intuition



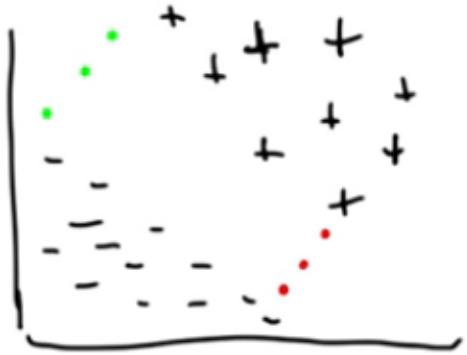
(Ref: How Classification Algorithms Work? - Thales Sehn Korting)

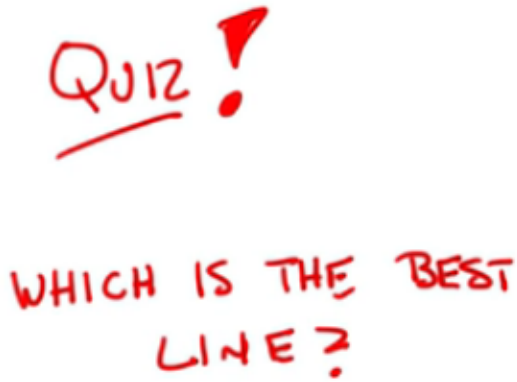
- For data points labeled as circles, when features are put into "g" equation, the result is more than equal to 1
- For data points labeled as squares, when features are put into "g" equation, the result is less than equal to -1

YHK

SVM Quick Derivation

Which is the best line?



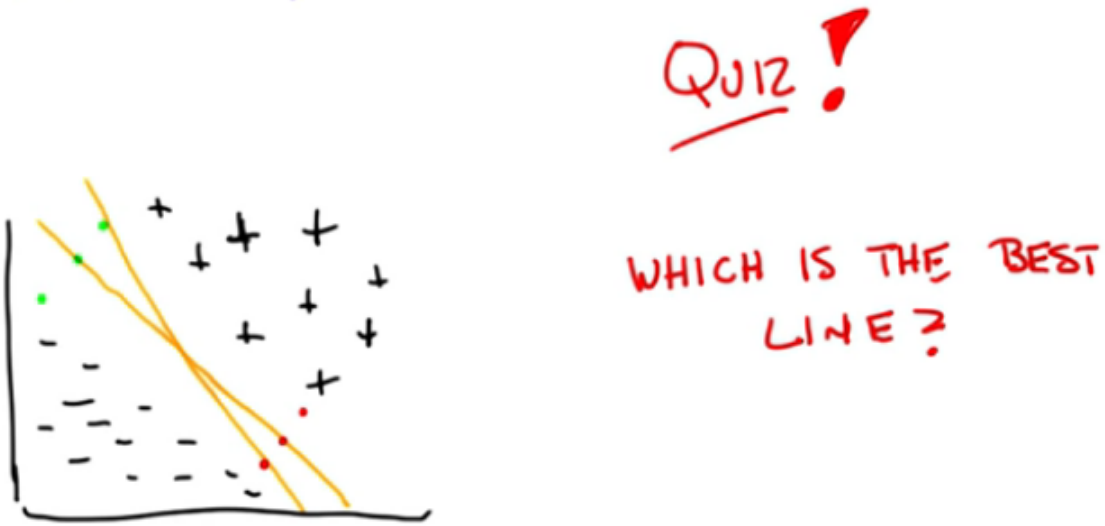


(Ref: Support Vector Machine - Georgia Tech - Machine Learning - Udacity)

- Question: Which is the best line?
- Not any random line!!
- There are 3 green dots and 3 red dots in the gap (gutter/margin), which 2 dots form best line?

YHK

Better Margin for Unseen points

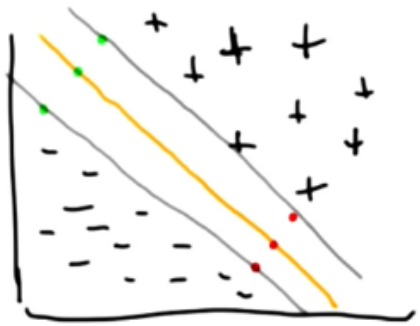


(Ref: Support Vector Machine - Georgia Tech - Machine Learning - Udacity)

- With 3 green forming lines with 3 reds each, there are 9 possible lines.
- And the ALL separate the positive and negative points.
- Intuitively, we thought that the middle line looks GOOD.
- The second, more slanting line is not good as its closer to some of the points than the middle line. Like Over-fitting.
- We need clear and BIG separation to avoid mis-classifications as much as possible. Because we don't know what un-seen points will bring. Better the gutter, better chances of good classification.

YHK

Maximum Margin

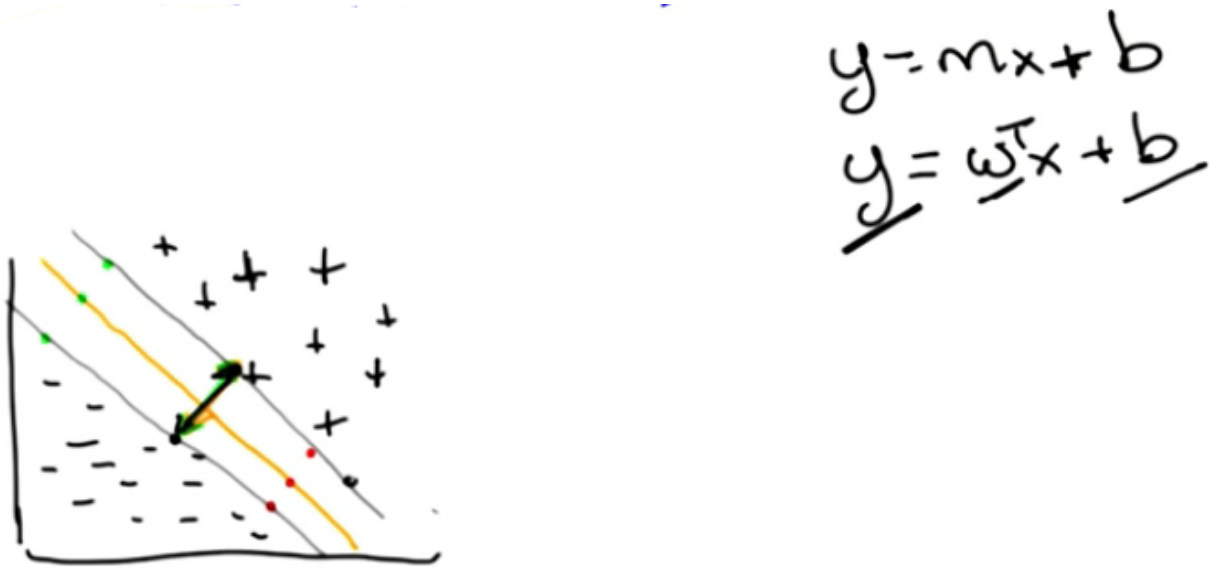


(Ref: Support Vector Machine - Georgia Tech - Machine Learning - Udacity)

- Even for parallel lines, the side lines are not good as they are CLOSE to either of the positive or negative lines.
- Top line is where the closest Positive line, bottom is to Negative line.
- Middle is the best as it has good space/margin around it.
- So, Goal is to find a separating plane that separates the classes as per the training data

YHK

Hyperplane with Boolean output



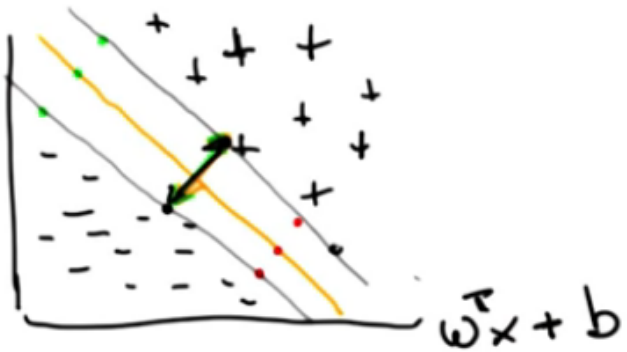
(Ref: Support Vector Machine - Georgia Tech - Machine Learning - Udacity)

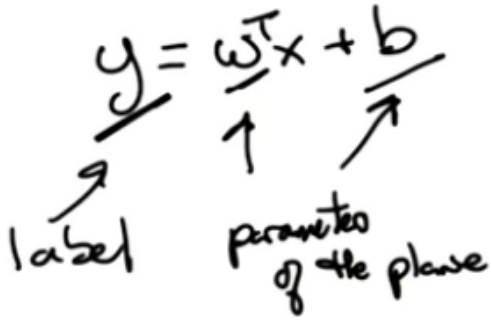
Maths revision: Equations:

- Equation of line is $y = mx + b$. If x has more than 2 dimensions, then this is equation of a plane, a hyper plane and then mx changes to $w^T X$, where w is slopes vector.
- Here y is not the coordinate, but boolean values.

YHK

Middle Hyperplane





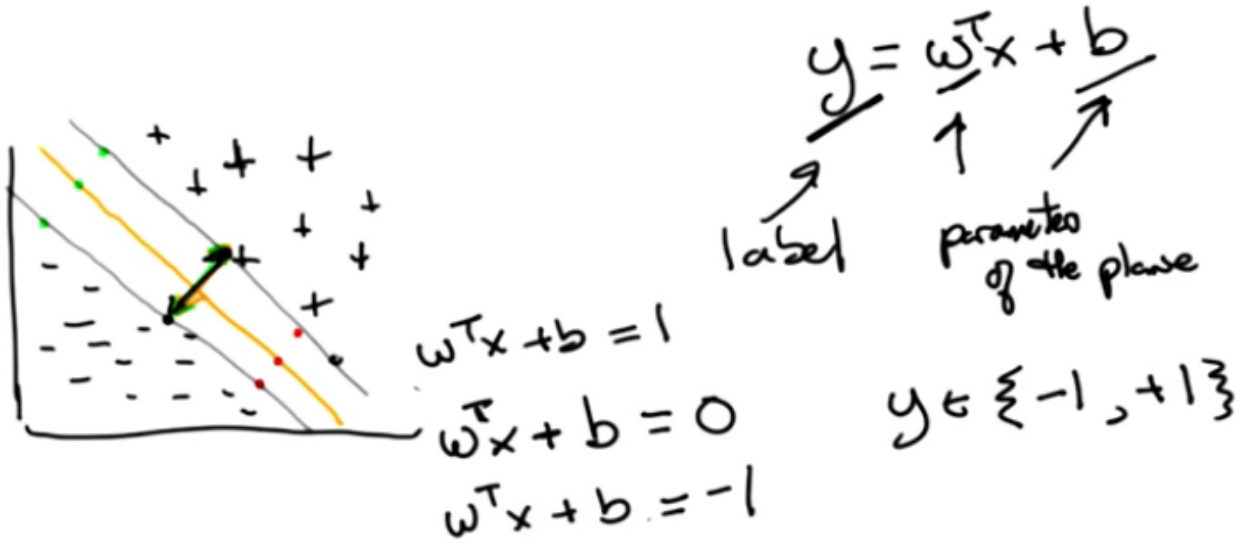
(Ref: Support Vector Machine - Georgia Tech - Machine Learning - Udacity)

Maths revision: Equations:

- y : classification label
- w^T : parameters, weights, coefficients
- b : bias
- Question: What would be output y for any point (x) on the middle line?

YHK

SVM Derivation

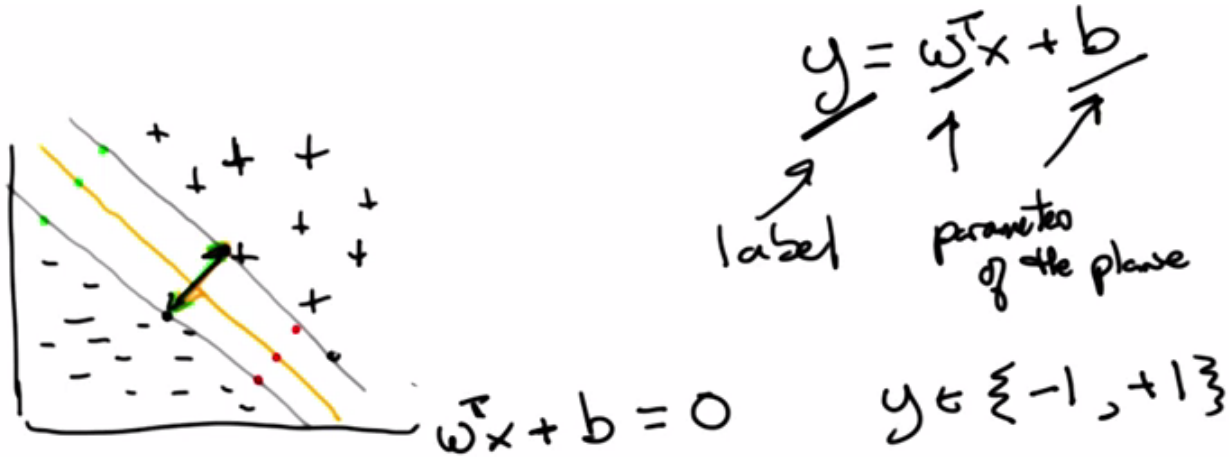


(Ref: Support Vector Machine - Georgia Tech - Machine Learning - Udacity)

- 0. As its neither positive nor negative.
- So, the equation of the middle line is $w^T X + b = 0$
- What are the equations of other boundary lines?
- So, lets make them -1 and $+1$, to denote negative and positive labels.

YHK

SVM Derivation

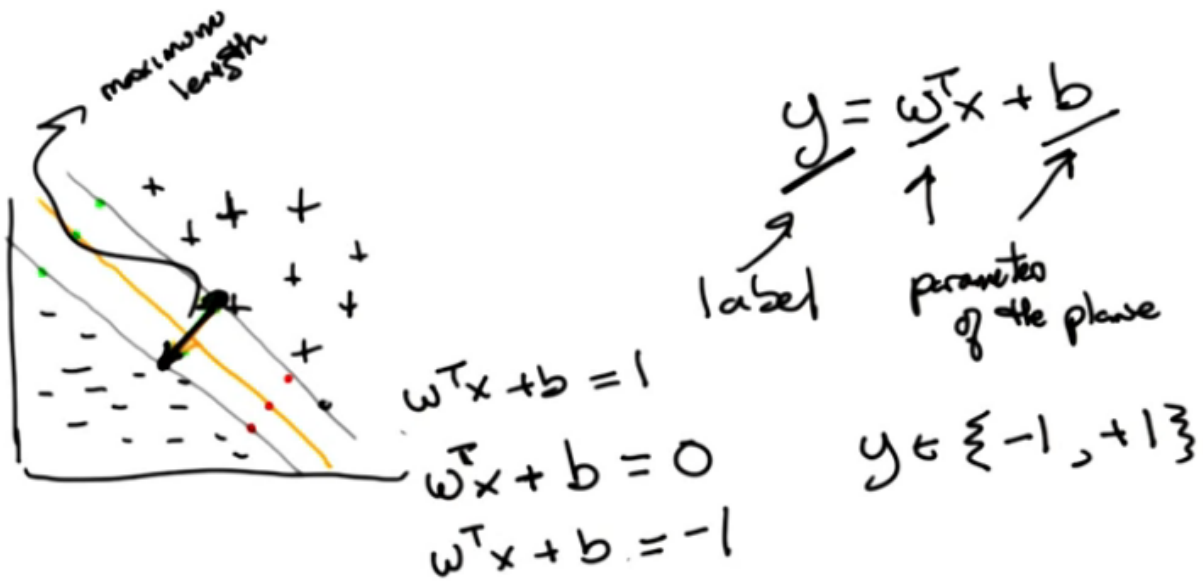


(Ref: Support Vector Machine - Georgia Tech - Machine Learning - Udacity)

- On the line $y = 0$ so eqn of the hyperplane is $\vec{w}^T \vec{x} + b = 0$
- For left support point, $y = -1$ so equation of left support boundary is $\vec{w}^T \vec{x} + b = -1$ And the full zone as ≤ -1
- For right support point, $y = +1$ so equation of right support boundary is $\vec{w}^T \vec{x} + b = +1$ And the full zone as $\geq +1$

YHK

SVM Derivation

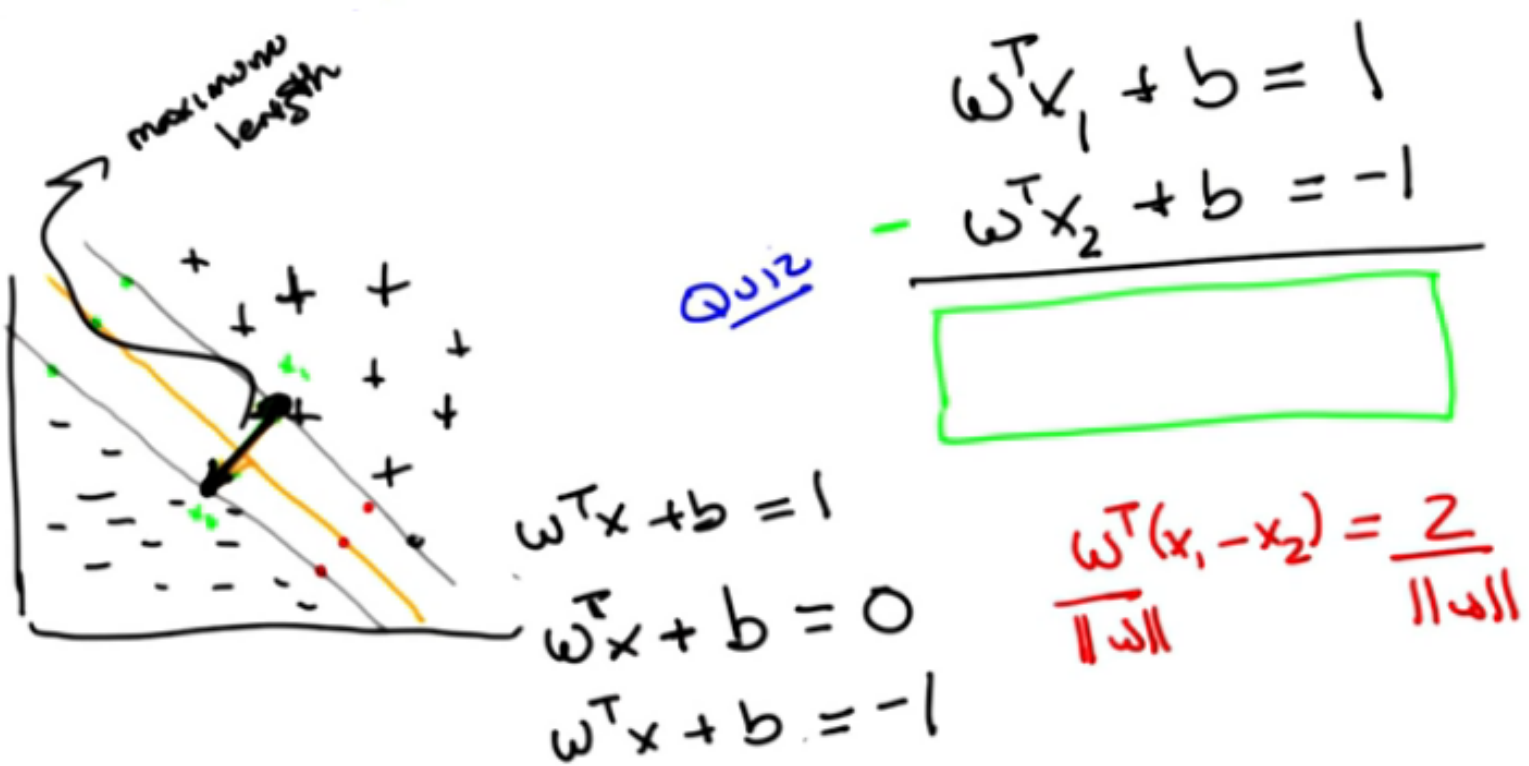


(Ref: Support Vector Machine - Georgia Tech - Machine Learning - Udacity)

- We want Margin/Width to be maximum. What is the equation of the Width?
- Can we find out, as the Width line is perpendicular to, to the boundary lines and its made of end points lying on the boundary lines?
- Lets call the points x_1 and x_2, so the vector diff between these is Width.

YHK

Equation of Width

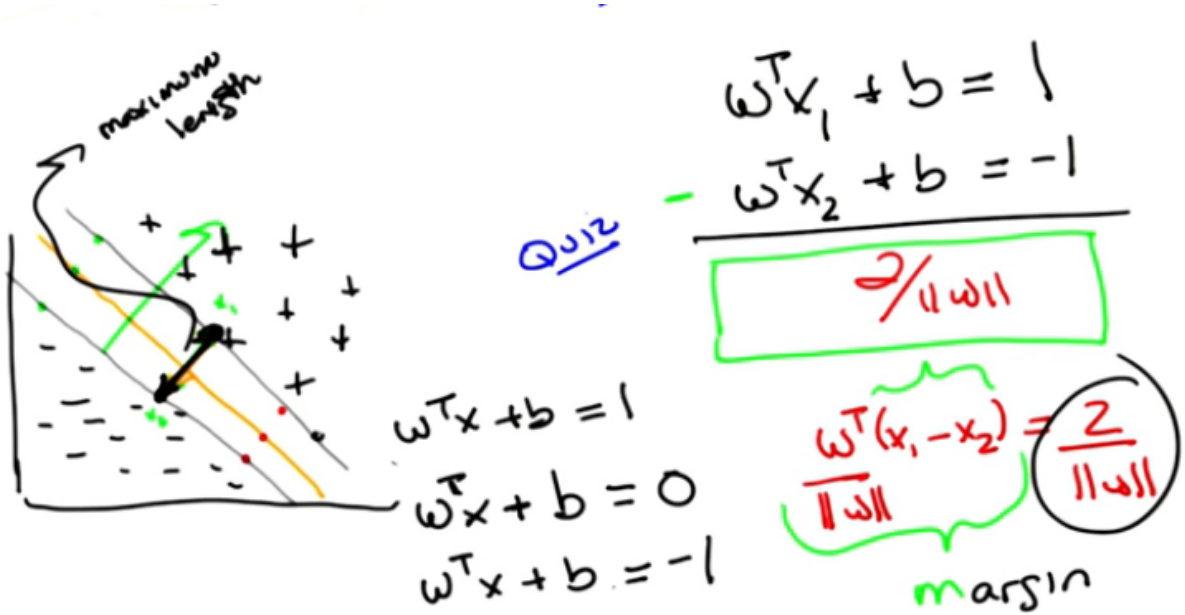


(Ref: Support Vector Machine - Georgia Tech - Machine Learning - Udacity)

- Two equations, 2 unknowns. Subtract. Result is $w^T(x_1 - x_2) = 2$
- Assume w is a number , you would divide 2 by it. But it is a vector. Why not take divide each side by length of w. $w/||w||$ is unit vector.
- So, on left hand side: $x_1 - x_2$ vector has been projected on unit w vector. Thats dot product, which is 2.

YHK

Maximizing Width



(Ref: Support Vector Machine - Georgia Tech - Machine Learning - Udacity)

- w is the direction along perpendicular to boundary lines, the width direction.
- So, we maximize width m ("margin") while classifying everything correctly (thats the constraint)
- But how to put this constraint mathematically?

YHK

Combining 2 Equations

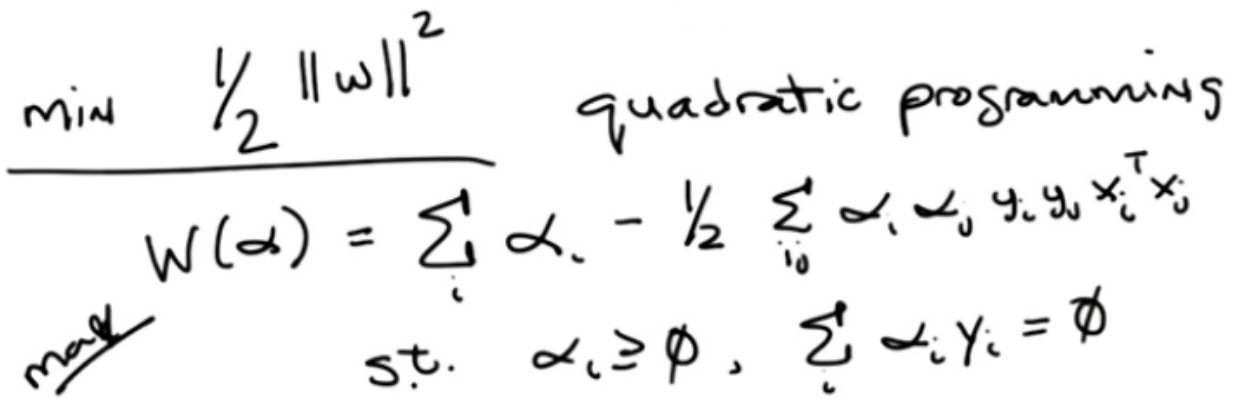
$$\max \frac{2}{\|w\|} \quad \text{while classifying everything correctly} \quad y_i(w^T x_i + b) \geq 1 \quad \forall_i$$

(Ref: Support Vector Machine - Georgia Tech - Machine Learning - Udacity)

- Lets combine both boundary lines equations into one, by leveraging the label y_i itself.
- $y_i(w^T X_i + b) \geq 1$. The "plus minus 1" thing on the right side has changed to just "plus 1" because y_i becomes minus if right hand side is minus, and both cancel out!!! Wonderful trick.

YHK

Hard Stuff, Leap of Faith



(Ref: Support Vector Machine - Georgia Tech - Machine Learning - Udacity)

- Instead of solving "max $\frac{2}{||w||}$ problem, is better to solve "min $\frac{1}{2}||w||^2$" problem. (Because lengths are positive and now we get good convex function to optimize, easier to compute by algorithm. Its a quadratic programming problem, and people know how to solve it. It has unique solution.)
- Another (not to be questioned!!) change is to convert the quadratic programming eqn to maximizing Lagrange Multiplier optimization form.
- Its sum of alphas (for all i data points) minus half time, every pair points, x and label(y) values, multiplied by own alphas; with constraints such as alphas are positives and the other constraint shown.

YHK

Just assume, This Works!!

$$W(\alpha) = \sum_i \alpha_i - \frac{1}{2} \sum_{ij} \alpha_i \alpha_j y_i y_j x_i^T x_j$$

max

$$\text{s.t.} \quad \alpha_i \geq 0, \quad \sum_i \alpha_i y_i = 0$$

$$w = \sum_i \alpha_i y_i x_i \;, \; b$$

α_i mostly 0 $\Rightarrow$ few of x_i matter

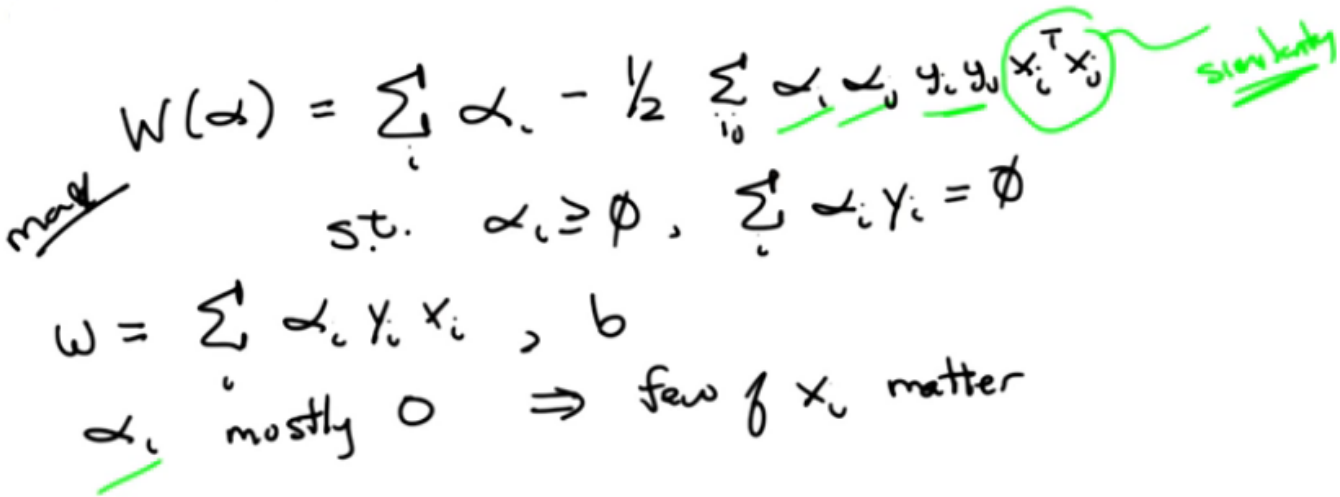
(Ref: Support Vector Machine - Georgia Tech - Machine Learning - Udacity)

Properties:

- Maximizing alphas, then W can be calculated the b can be calculated.
- Lets make small w is the 2nd term.
- Most of the alphas are 0. Means only few points are part of the summation. The points for which alphas are non-zero are called support vectors. They are the points on the boundary lines.
- So, this is the Machine that needs only Support Vectors.

YHK

SVM Derivation Closure



(Ref: Support Vector Machine - Georgia Tech - Machine Learning - Udacity)

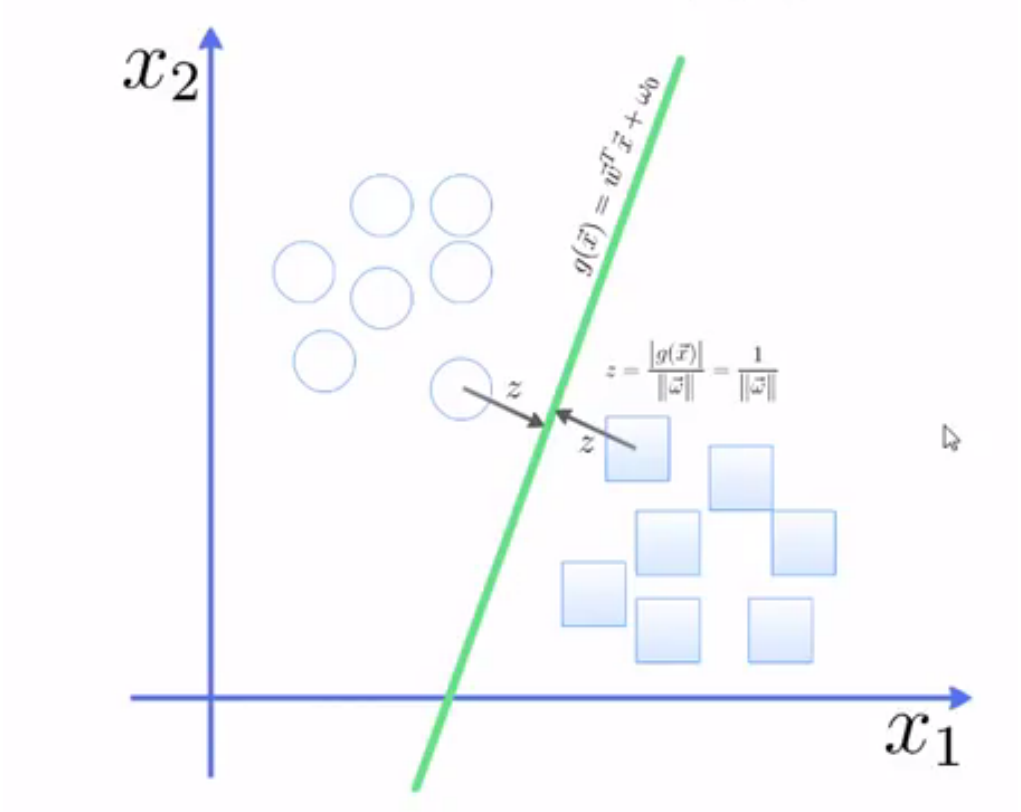
Properties:

- Points away DO NOT MATTER. Its like Nearest Neighbor where only local points matter!! But here K is not provided, but we already know support points (and thus their count, ie K) by quadratic programing
- The x product terms, are the dot products. If both are in same line, its a large number, but if they are perpendicular its towards 0. Its similarity.
- So, find all support points, and from that keep only collinear like points.
- This maximizes W, and thus the width.

YHK

Back to SVM Intuition, old symbols

SVM Intuition

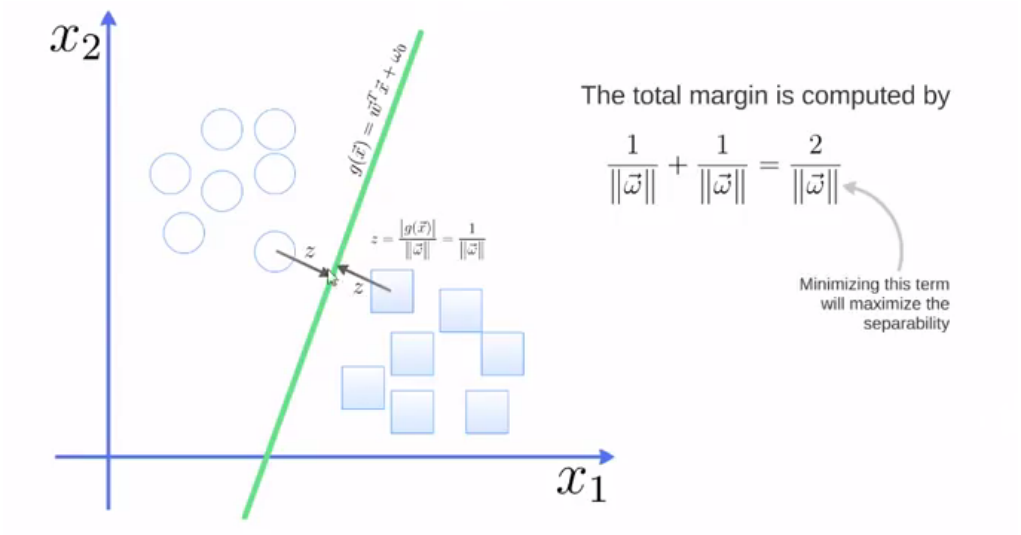


(Ref: How Classification Algorithms Work? - Thales Sehn Korting)

- Distance of a point from a hyper-plane is z
- Given by a formula $z = \frac{|g(x)|}{||w||}$, is also called 'margin'

YHK

SVM Intuition



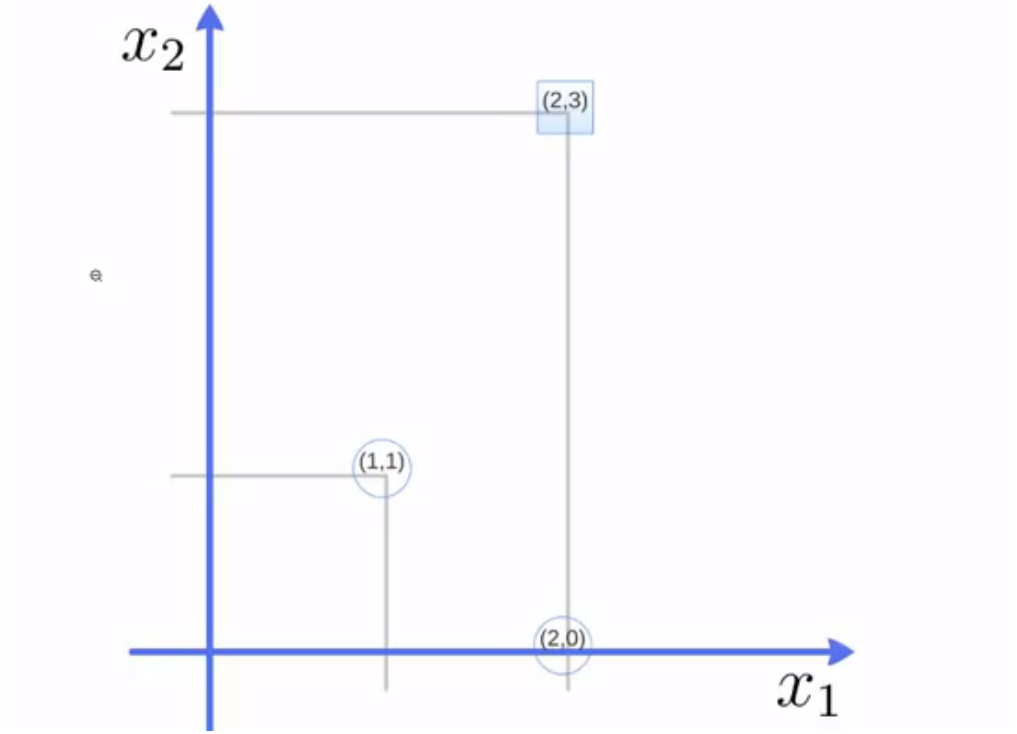
(Ref: How Classification Algorithms Work? - Thales Sehn Korting)

- For any point, $g(\overrightarrow{x})$ is atleast "1", on either side (this is the minimum-est, g(x) can get, from the formula)
- So min distance on either side is $\frac{1}{||\overrightarrow{w}||}$
- Total margin from both sides is $\frac{2}{||\overrightarrow{w}||}$
- So, if we minimize $\overrightarrow{w}$, then margin will be maximum.

YHK

SVM Example

SVM Example

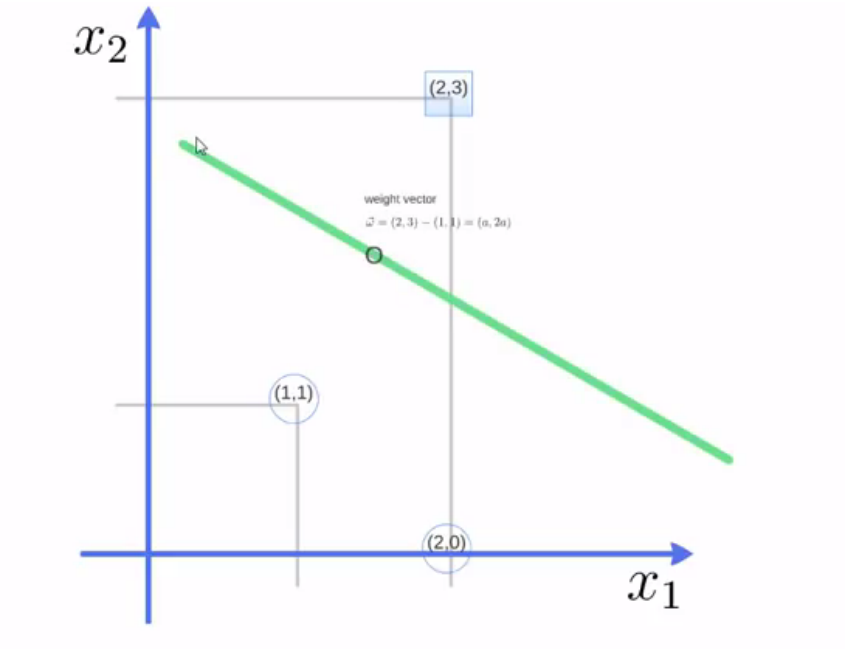


(Ref: How Classification Algorithms Work? - Thales Sehn Korting)

- Lets say we have just 3 points, 2 circles ([1,1] and [2,0]) and one square [2,3]
- We want to design SVM classifier, ie the Hyper-plane

YHK

SVM Example



(Ref: How Classification Algorithms Work? - Thales Sehn Korting)

- We know that the hyper-plane line would be equidistant from two closest points (2,3) and (1,1)
- So, generically the weight vector would be of the form $\bar{w} = (2, 3) - (1, 1) = (a, 2a)$
- Need to find a and then $\bar{w}, w_0$

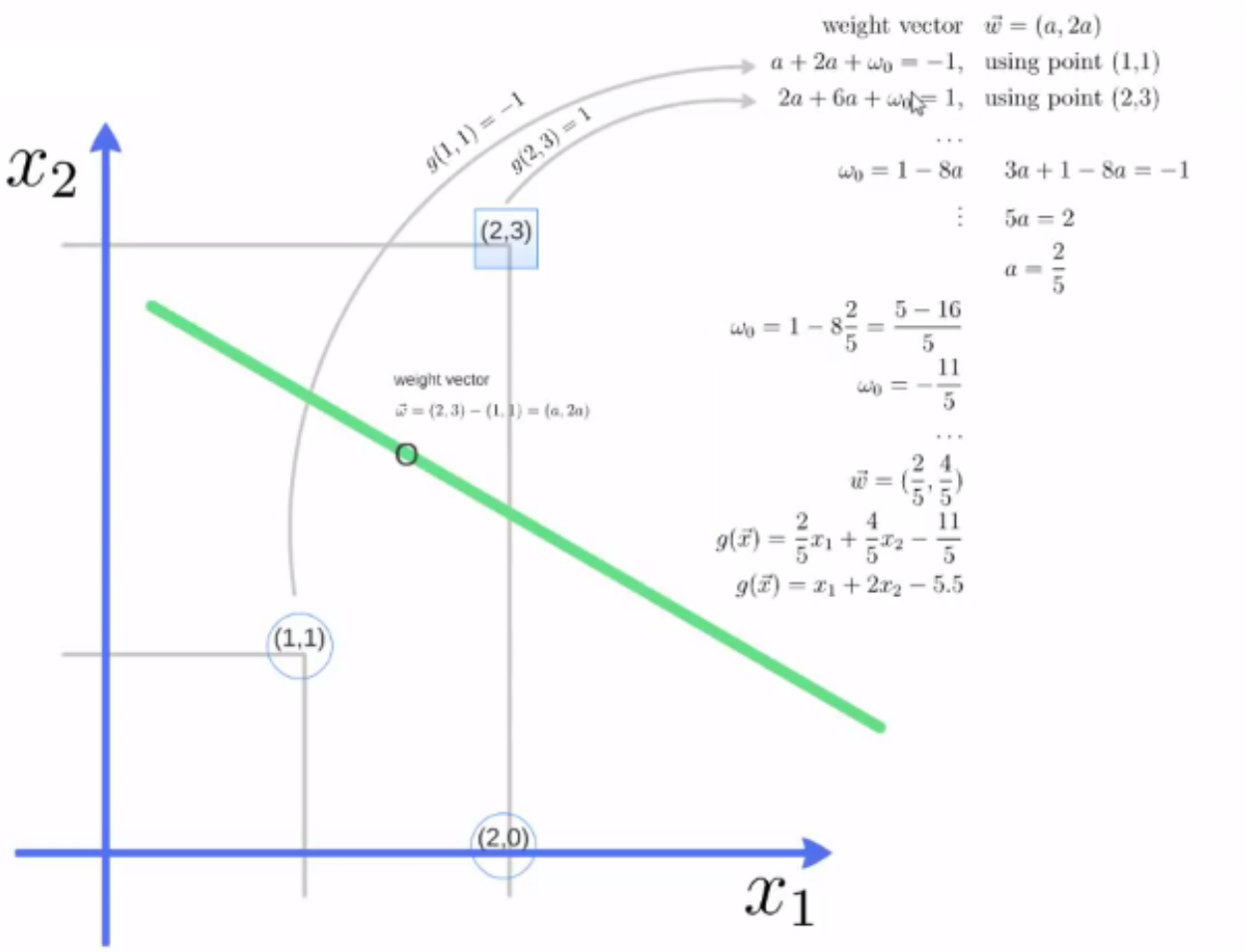
YHK

SVM Example

- weight vector $\bar{w} = (a, 2a)$
- Using Negative point (1,1), putting in equation $g(\bar{x}) = \bar{w}x + w_0 = y$ is $g(1,1) = (a, 2a).(1,1) + w_0 = -1$, is $a + 2a + w_0 = -1$
- Using Positive point (2,3), putting in equation $g(\bar{x}) = \bar{w}x + w_0 = y$ is $g(2,3) = (a, 2a).(2,3) + w_0 = 1$, is $2a + 6a + w_0 = 1$
- Solving these two equations simultaneously. From the positive-point equation, $w_0 = 1 - 8a$. Putting it in the negative-point equation: $3a + 1 - 8a = -1$, thus $5a = 2$ so $a = \frac{2}{5}$
- Putting a back, $w_0 = 1 - 8\frac{2}{5} = -\frac{11}{5}$
- Thus , $\bar{w} = (\frac{2}{5}, \frac{4}{5})$
- Putting in original Hyperplane equation $g(\bar{x}) = \frac{2}{5}x_1 + \frac{4}{5}x_2 - \frac{11}{5}$
- Being equal to 0 for the middle line, it can be simplified to $g(\bar{x}) = x_1 + 2x_2 - 5.5$

YHK

SVM Example

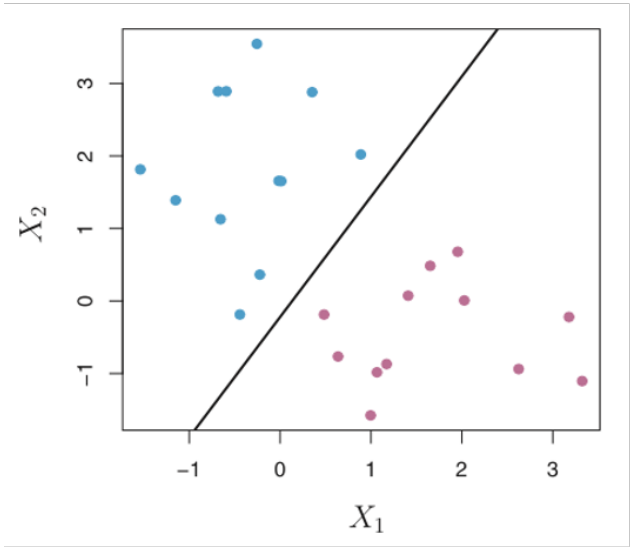


(Ref: How Classification Algorithms Work? - Thales Sehn Korting)

YHK

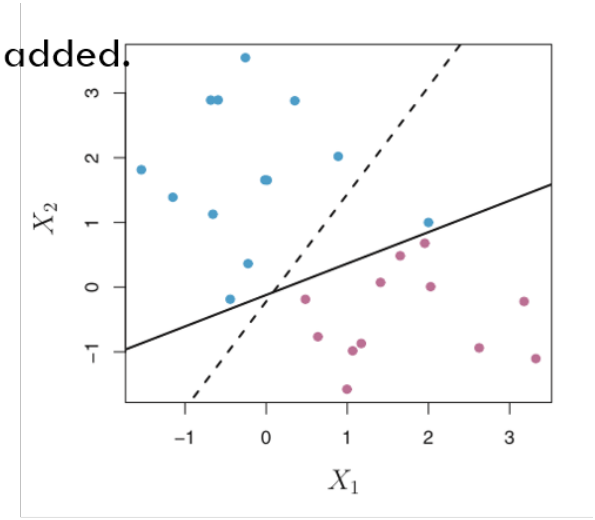
SVM Aspects

New point added



Maximum margin classifier.
Perfectly segments training data.

New data point added.

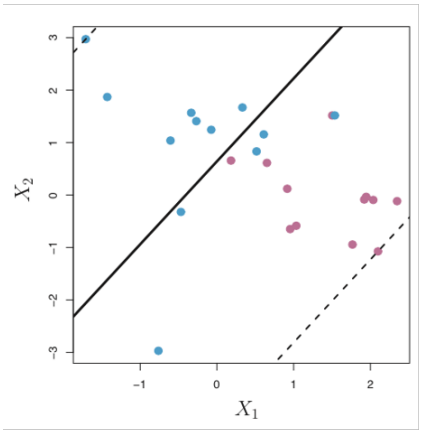


Dramatic shift in maximal margin hyperplane.
Model has *high variance* when trying to maintain perfect segmentation.

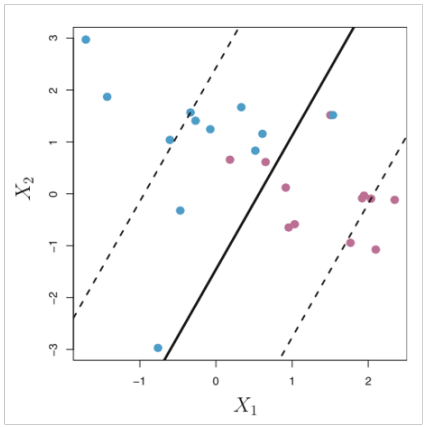
YHK

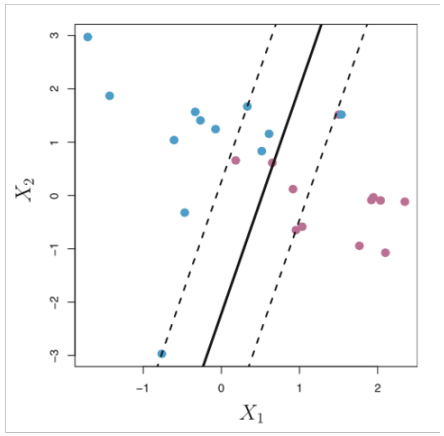
Constructing the Support Vector Classifier

- How much "softness" (mis-classifications) is ideal?
- Specification of non-negative tuning parameter C
  - Generally chosen by analyst following cross-validation
  - Large C: wider margin; more instances violate margin
  - Small C: narrower margin; less tolerance for instances that violate margin



Lower variance.

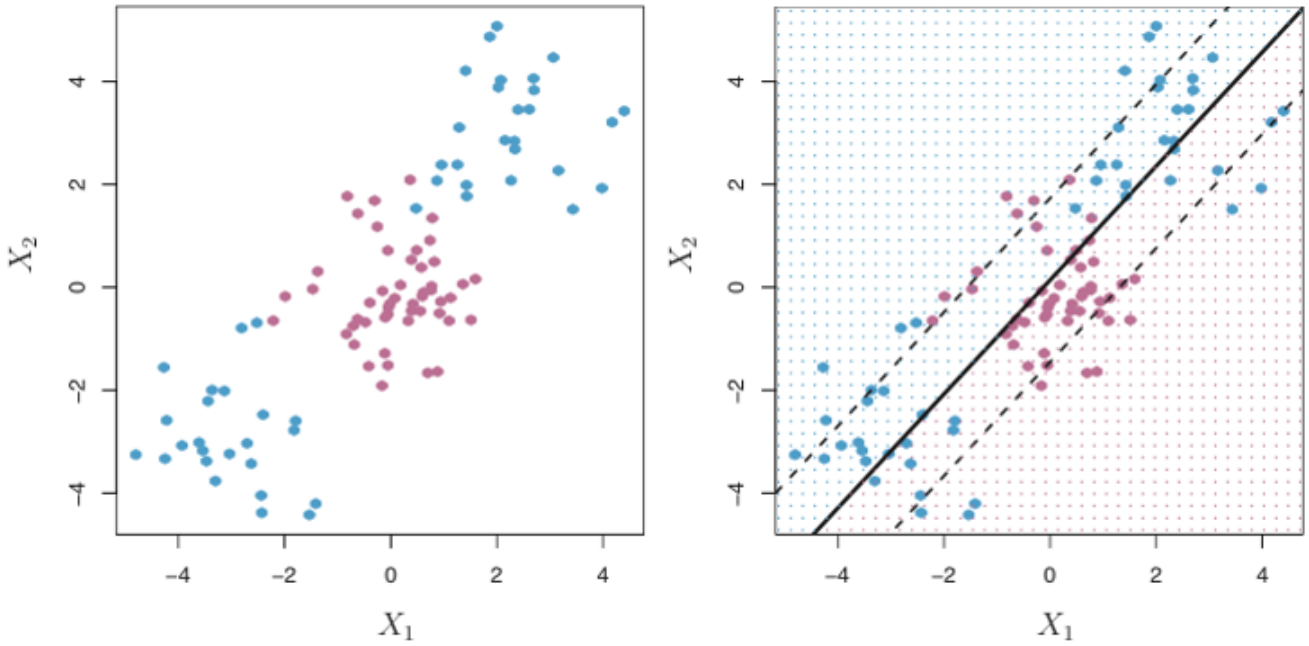




Higher variance.

Non-linear decision boundary?

What if a non-linear decision boundary is needed?

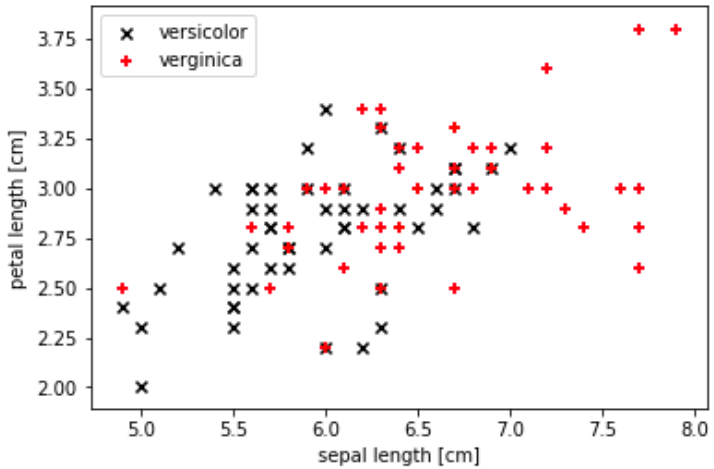


Poor performance using this decision boundary. Can we twist the boundary? Or Can we twist the data?

YHK

Non-linear Dataset

Non-linear data set are difficult to be separated using a linear hyperplane. SVM algorithm is related to finding the hyperplane which separates the data based on maximum margin.
E.g. Iris dataset



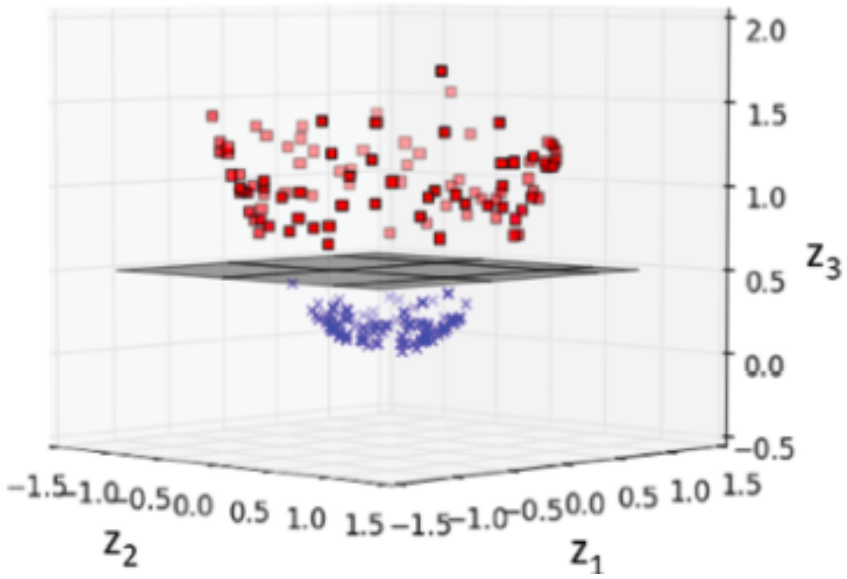
The above data set is in 2-dimensions. And, it is not linearly inseparable in a smooth manner.

(Ref: Machine Learning – SVM Kernel Trick Example - Ajitesh Kumar)

YHK

Non-linear Dataset

Can the data set be represented in the following form by adding another dimension? If it is possible, a hyperplane can be found which separates them clearly.



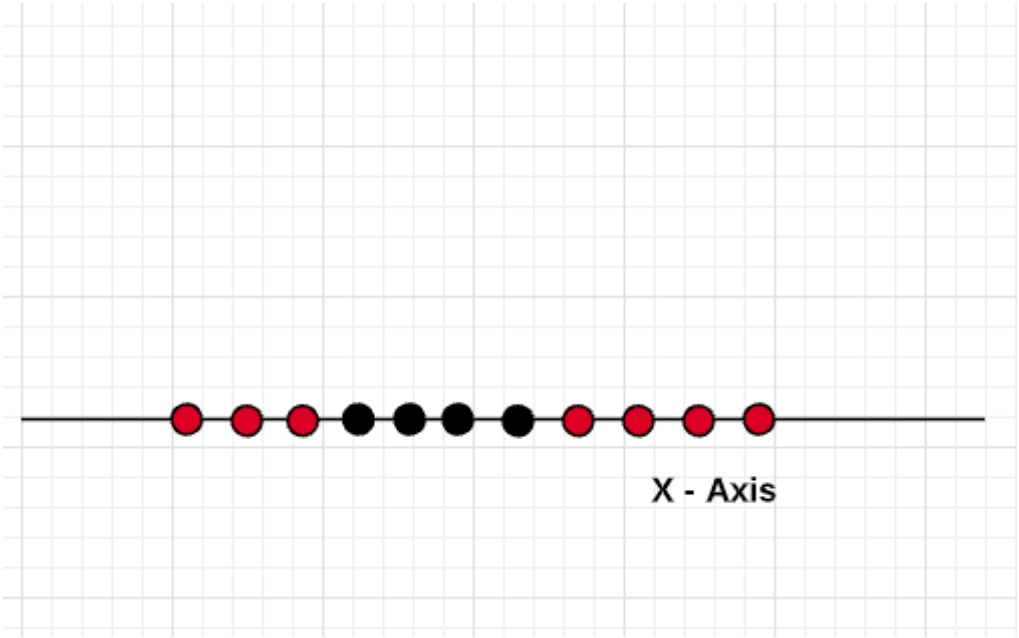
This is where the kernel method comes to the rescue. The idea is to apply a function that projects / transform the data in such a manner that the data becomes linearly separable. In case of SVM algorithm, data becomes linearly separable by applying maximum margin.

(Ref: Machine Learning – SVM Kernel Trick Example - Ajitesh Kumar)

YHK

Simple Example

1-D: not easy to separate and how adding another dimension makes it easy.

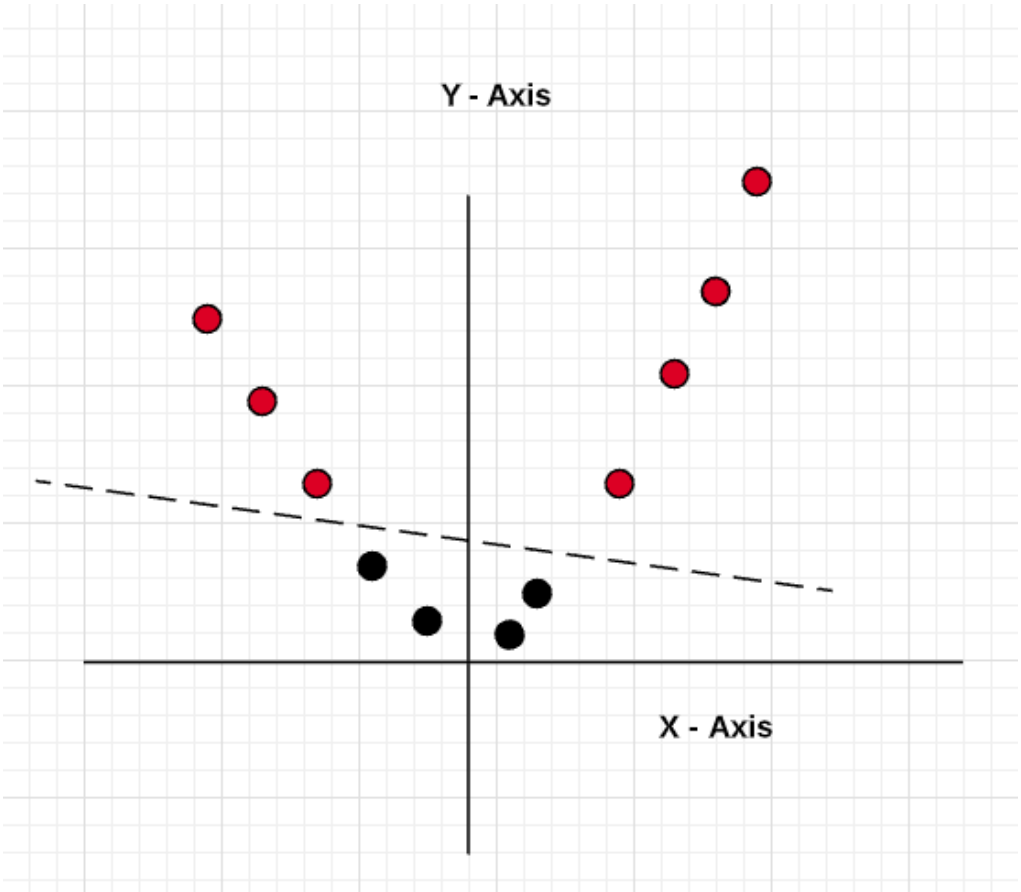


(Ref: Machine Learning – SVM Kernel Trick Example - Ajitesh Kumar)

YHK

Simple Example

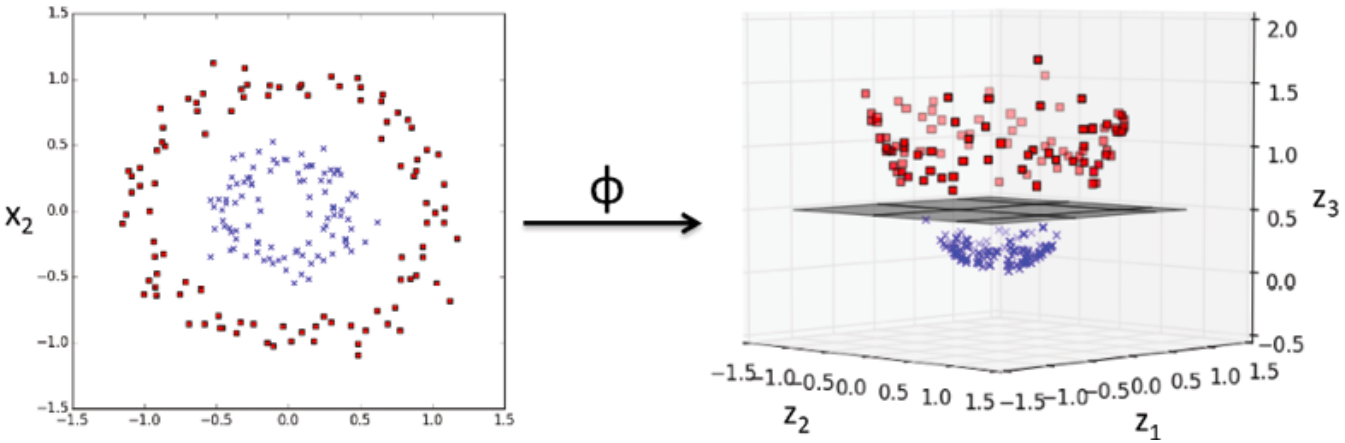
Let's apply the method of adding another dimension to the data by using the function $Y = X^2$ (X-squared). Thus, the data looks like the following after applying the kernel function ($Y = X^2$) and becomes linearly separable.



(Ref: Machine Learning – SVM Kernel Trick Example - Ajitesh Kumar)

What are Kernel Methods?

- The idea is to create nonlinear combinations of the original features to project them onto a higher-dimensional space via a mapping function $\phi(x)$, where the data becomes linearly separable.
- In the diagram given below, the two-dimensional dataset (X1, X2) is projected into a new three-dimensional feature space (Z1, Z2, Z3) where the classes become separable.



(Ref: Machine Learning – SVM Kernel Trick Example - Ajitesh Kumar)

YHK

What are Kernel Methods?

- Kernel trick is to convert dot product of support vectors to the dot product of mapping function.
- Recall the optimization problem for SVM:

$$\sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \overrightarrow{x_{SVi}} \cdot \overrightarrow{x_{SVj}}$$

- Pay attention to the pink circle which represents the dot product of the support vectors. The trick is to replace this inner product with another one without even knowing ϕ

$$\sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \ \phi(\overrightarrow{x_{SVi}}) \cdot \phi(\overrightarrow{x_{SVj}})$$

- From above, if there is a function that can be represented as an inner product of mapping function, all we will be required to know is that function and not the mapping function. That function is called a Kernel Function.

(Ref: Machine Learning – SVM Kernel Trick Example - Ajitesh Kumar)

YHK

What are Kernel Functions and its different types?

- The kernel function is a function which can be represented the dot product of the mapping function (kernel method) and can be represented as the following:

$$K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$$

- Kernel function reduces the complexity of finding the mapping function. So, the kernel function defines the inner product in the transformed space. The following are different kinds of kernel functions.
  - Polynomial kernel
  - Gaussian kernel
  - Radial basis function (RBF) kernel
  - Sigmoid kernel

(Ref: Machine Learning – SVM Kernel Trick Example - Ajitesh Kumar)

YHK

SVM - Multiclass

Multiclass Problems

- Scenario: target class is more than 2 categories
- How to extend binary classifiers to handle multi-class problems?

YHK

#1 - Multiclass: One-Against-Rest

- Assume multi-class data-set with K target classes
- Decompose into K binary problems
- Idea: For each target class yi create a single binary problem, with classifier Ci: Class yi: positive ; All other classes: negative;
- Training: use all instances; each instance used in training each of the Ci classifiers;
- Testing: run testing instance through each classifier; record votes for each yi class (negative prediction is a vote for all other classes); class with most votes is the predicted class

YHK

#2 - Multiclass: One-Against-One

- Assume multi-class data-set with K target classes
- Train K(K-1)/2 binary classifiers (many more than One-Against-Rest)
- Idea: Each classifier distinguishes between pair of classes (yi, yj); Classifier ignores records that don't belong to y_i or y_j
- Training: use all instances; each instance only used in training "relevant classifiers" (K of them)
- Testing: run testing instance through each classifier; record votes for each y_i class; class with most votes is the predicted class;

YHK

Quick Check: Support Vector Machine

Think About It

If you added a new training point that lies far from the decision boundary, on the correct side of the margin, would the SVM's maximum-margin hyperplane change? What does your answer tell you about how much of the training set actually "matters" to an SVM?

YHK

Quick Check: Support Vector Machine

Answer

No – the hyperplane depends only on the support vectors (the points sitting on or violating the margin). A point that's already correctly classified with room to spare gets an alpha of 0 in the dual solution, so adding or removing it changes nothing. This is why SVMs can generalize well from relatively few "boundary case" examples – most of the training data turns out to be redundant once the support vectors are known.

YHK

SVM (Recap)

- Maximum Margin Classifier is natural way to perform classification if a separating hyper-plane exists.
- This generalization is called: support vector classifier
- In many cases, no separating hyper-plane will exist. So need to loosen a bit.
- Maximal Margin Classifier: no training errors allowed
- Support Margin Classifier: tolerate training errors. Approach: Soft margin
- Will allow construction of linear decision boundary even when classes are not linearly separable. Kernel Tricks.

YHK

Thanks . . .

- Office Hours: Saturdays, 3 to 5 pm (IST); Free-Open to all; email for appointment to yogeshkulkarni at yahoo dot com
- Call + 9 1 9 8 9 0 2 5 1 4 0 6

(https://www.linkedin.com/in/yogeshkulkarni/)

(https://medium.com/@yogeshharibhaukulkarni)

(https://www.github.com/yogeshhk/)

YHK